Introduction to Data Science

Session 2: Programming I

Simon Munzert
Hertie School | GRAD-C11/E1339

Table of contents

Recap: tidyverse basics

What is the tidyverse?

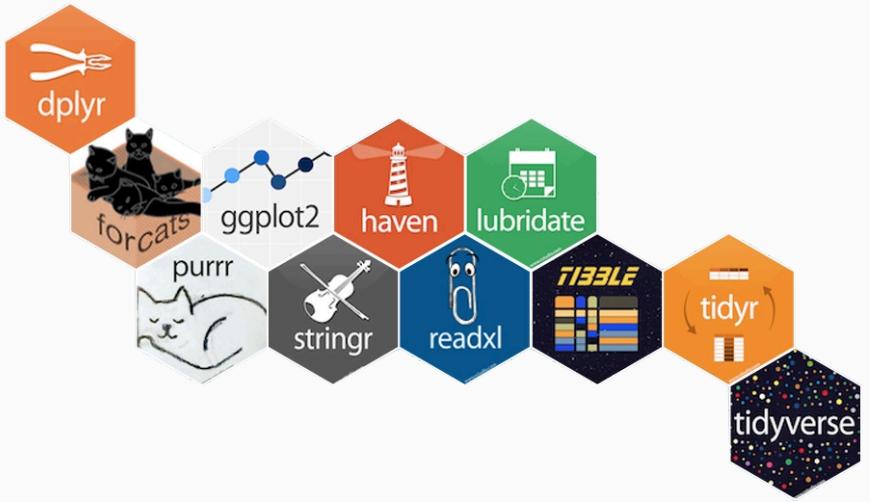
R packages for data science

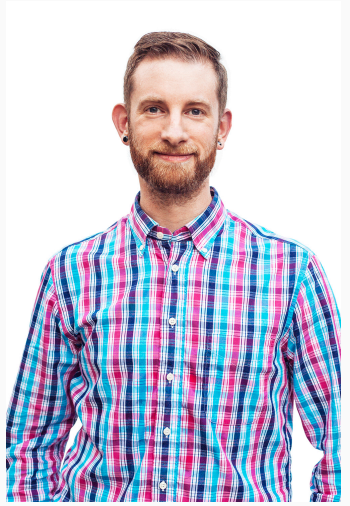
- Let's take it from the tidyverse website:

"The tidyverse is an opinionated collection of R packages designed for data science. All packages share an underlying design philosophy, grammar, and data structures."

- It's the contribution of many people of the R community.
- Hadley Wickham had a key role in shaping it by developing many of the core packages, such as `ggplot2`, `dplyr`, `tidyr`, `tibble`, and `stringr`.
- Install the complete tidyverse with:

```
R> install.packages("tidyverse")
```



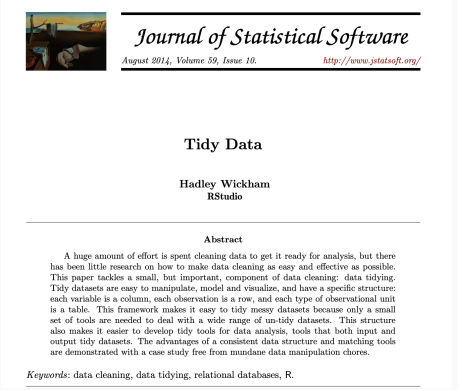


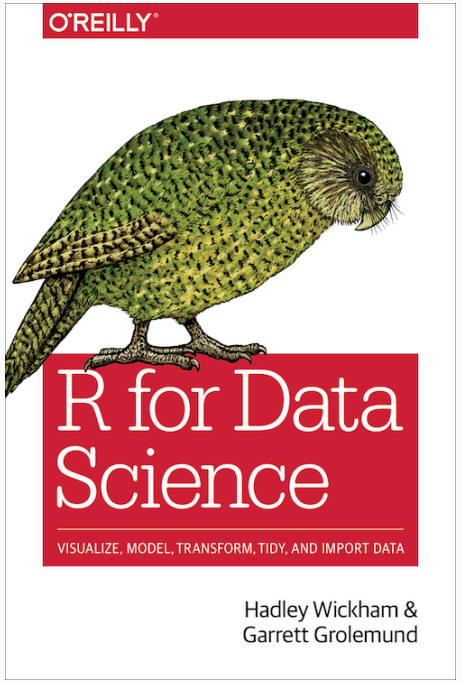
Hadley Wickham

A guide to the tidyverse

Valuable resources

- Welcome to the Tidyverse, a quick overview from many tidyverse contributors
- Tidy data, a foundational paper on data wrangling and structuring, by Hadley Wickham, 2014, *Journal of Statistical Software*; check here for a hands-on vignette based on the `tidyr` package
- The tidyverse design guide, a (soon-to-be book) manifesto to promote design consistency across the tidyverse
- R for Data Science, our main textbook for this course





Tidyverse packages

Loading the tidyverse

```
R> library(tidyverse)
```

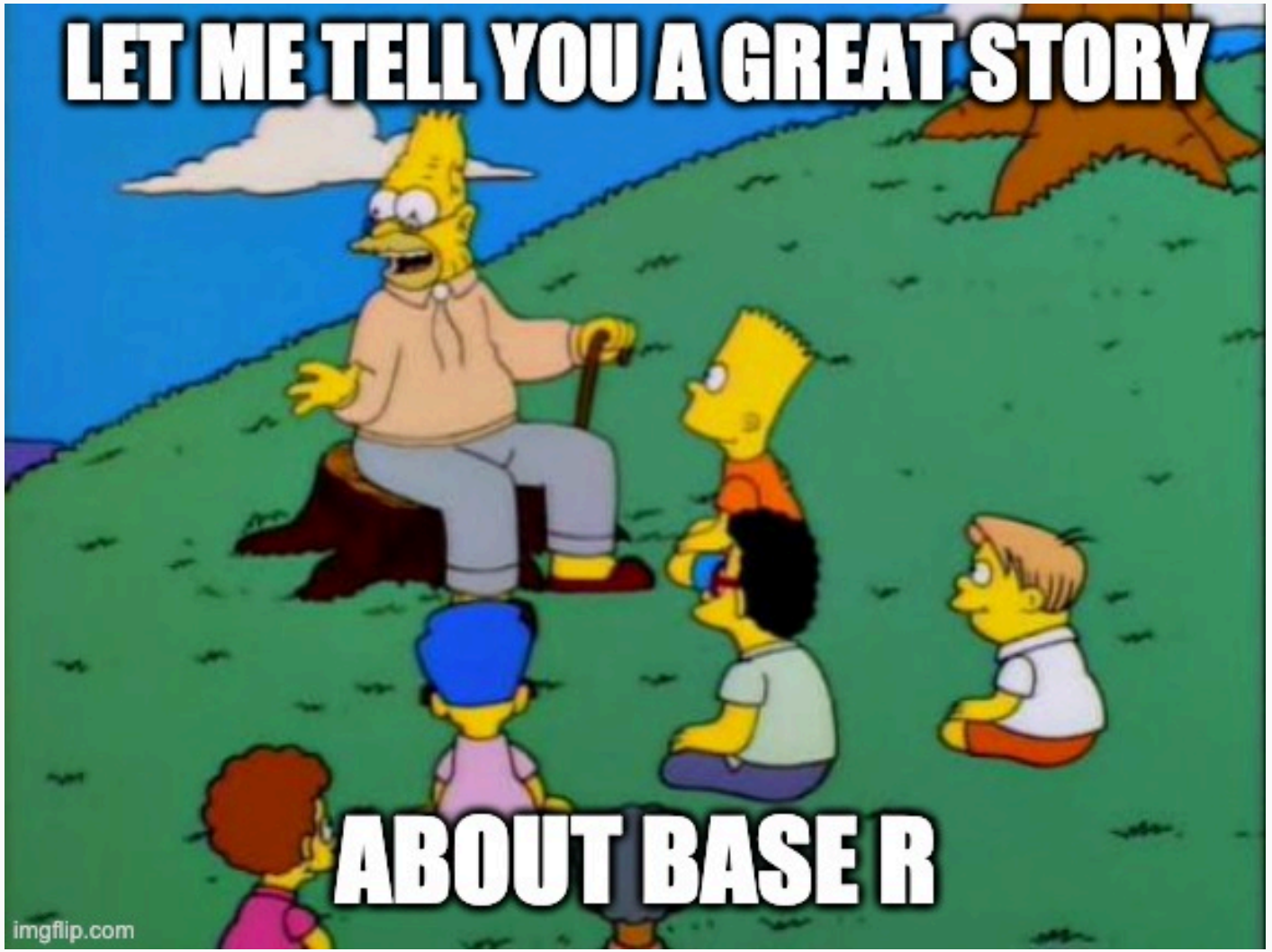
- In addition to the currently 8 core packages, the tidyverse includes many others for more specialized usage.[1]
- We'll use several of these additional packages during the remainder of this course (e.g., the `rvest` package for web scraping). Bear in mind that these packages will have to be loaded separately.
- See here for an overview, or just in R directly:

```
R> tidyverse_packages()
```

```
 [1] "broom"        "conflicted"   "cli"          "dbplyr"       "dplyr"        "dtplyr"       "forcats"
 [8] "ggplot2"      "googledrive"  "googlesheets4" "haven"        "hms"          "httr"         "jsonlite"
[15] "lubridate"    "magrittr"     "modelr"       "pillar"       "purrr"        "ragg"         "readr"
[22] "readxl"       "reprex"       "rlang"        "rstudioapi"   "rvest"        "stringr"      "tibble"
[29] "tidyr"        "xml2"         "tidyverse"
```

[1] It also includes a *lot* of dependencies upon installation. This is a matter of some controversy.

Tidyverse vs. base R



Tidyverse vs. base R: what's the difference?

- Both are compatible. You can wrangle your data with `dplyr`, plot it with `ggplot2`, and model it with yet another package.
- Ultimately, the tidyverse is just a bunch of (hugely popular!) packages that share design principles.
- Often, tidyverse packages don't reinvent the wheel. Instead, they offer more consistency in naming, arguments, and output (among other things).
- For instance, compare function naming principles (`tidyverse::snake_case` vs `base::period.case` rule; more on these conventions later) in these examples:

tidyverse	base
`?readr::read_csv`	`?utils::read.csv`
`?dplyr::if_else`	`?base::ifelse`
`?tibble::tibble`	`?base::data.frame`

- If you call up the above examples, you'll see that the tidyverse alternative typically offers some enhancements or other useful options (and sometimes restrictions) over its base counterpart.
- And **remember:** There are (almost) always multiple ways to achieve a single goal in R.

Tidyverse vs. base R: what's the difference? *cont.*

Tidyverse

`Credit` sawiki.com

Base R

`Credit` multimedialab.be

Tidyverse vs. base R: what to use?

Stories from the past

- When I started to learn R ~16 years ago, there was no tidyverse. The learning curve felt much steeper. I often switched back to Stata for data wrangling.
- As the tidyverse grew, R became more convenient to use for the entire research pipeline.
- There's simply no need for you to live through the same pain.

Why we start with the tidyverse

- Because clever people think it's the right way.
- Documentation + community support are great.
- Having a consistent syntax makes it easier to learn.

You still will want to check out base R alternatives later

- Base R is extremely flexible and powerful (and stable).
- There are some things that you'll have to venture outside of the tidyverse for.
- A combination of tidyverse and base R is often the best solution to a problem.
- Excellent base R data manipulation tutorial here.

Functions

Tidy programming basics

"Tidy programming" is not a strictly defined practice in the tidyverse. However, there are some common programming strategies that help you keep your code and workflow tidy. These include:

- Pipes (you already learned how to use them ✅)
- User-generated functions
- Functional programming with `purrr`

The latter two are extremely helpful - in particular when you are confronted with iterative tasks.

We will now learn the basics of creating your own functions and functional programming with R. There is much more to learn about these topics, so we will revisit them as the course progresses.

Functional programming

R is a functional programming (FP) language. As Hadley Wickham puts it in Advanced R:

> This means that it provides many tools for the creation and manipulation of functions. In particular, R has what's known as first-class functions. You can do anything with functions that you can do with vectors: you can assign them to variables, store them in lists, pass them as arguments to other functions, create them inside functions, and even return them as the result of a function.

R encourages you to use and build your own functions to solve problems. Often, this implies decomposing a large problem into small pieces, and solving each of them with independent functions.

There is much more to learn about functions and functional programming. Useful resources include:

- The chapter on functions in R for Data Science.
- The section on functional programming in Advanced R.
- The R packages book. In a way, bundling functions in a package is sometimes the next logical step.

Creating functions

Why creating functions?

That's a legit question. There are 22,000+ **packages** on CRAN (and many, many more on GitHub and other repositories) containing zillions of functions. Why should you create yet another one?

- Every data science project is unique. There are problems only you have to solve.
- For problems that are repetitive, you'll quickly look for options to automate the task.
- Functions are a great way to automate.

Examples where creating functions makes sense

1. You want to scrape thousands of websites. This implies multiple steps, from downloading to parsing and cleaning. All these steps can be achieved with existing functions, but the fine-tuning is specific to the set of websites. You build one (or a set of) scraping functions that take the websites as input and return a cleaned data frame ready to be analyzed.

2. You want to estimate not one but multiple models on your dataset. The models vary both in terms of data input and specification. Again, based on existing modeling functions you tailor your own, allowing you to run all these models automatically and to parse the results into one clean data frame.

Basic syntax

Writing your own function in R is easy with the `function()` function[1]. The basic syntax is as follows:

```
R> my_func ← function(ARGUMENTS) {
+     OPERATIONS
+     return(VALUE)
+   }
```

[1] Yes, a function to create functions. 🤯

Basic syntax

Writing your own function in R is easy with the `function()` function[1]. The basic syntax is as follows:

```
R> my_func ← function(ARGUMENTS) {
+     OPERATIONS
+     return(VALUE)
+   }
```

- We write functions to apply them later. So, we have to give them a name. Here, we name it "`my_func`".
- Also, our function (almost) always needs input, plus we want to specify how exactly the function should behave. We can use arguments for this, which are specified as arguments of the `function()` function.

[1] Yes, a function to create functions. 🤯

Basic syntax

Writing your own function in R is easy with the `function()` function[1]. The basic syntax is as follows:

```
R> my_func ← function(ARGUMENTS) {
+     OPERATIONS
+     return(VALUE)
+   }
```

- Next, we specify anything we want the function to do.
- This comes in between curly brackets, `{ ... }`.
- Importantly, we can recycle arguments by calling them by their name.

[1] Yes, a function to create functions. 🤯

Basic syntax

Writing your own function in R is easy with the `function()` function[1]. The basic syntax is as follows:

```
R> my_func ← function(ARGUMENTS) {
+     OPERATIONS
+     return(VALUE)
+   }
```

- Finally, we specify what the function should return.
- This could be a list, data.frame, vector, sentence - or anything else really.
- Note that R automatically returns the final object that is written (not: assigned!) in your function by default. Still, my recommendation is that you get into the habit of assigning the return object(s) explicitly with `return()`.

[1] Yes, a function to create functions. 🤯

Basic syntax

Writing your own function in R is easy with the `function()` function[1]. The basic syntax is as follows:

```
R> my_func ← function(ARGUMENTS) {
+     OPERATIONS
+     return(VALUE)
+   }
```

- Oh, and don't forget to close the curly brackets...

[1] Yes, a function to create functions. 🤯

Basic syntax

Writing your own function in R is easy with the `function()` function[1]. The basic syntax is as follows:

```
R> my_func ← function(ARGUMENTS) {
+     OPERATIONS
+     return(VALUE)
+   }
```

Let's try it out with a simple example function - one that converts temperatures from Fahrenheit to Celsius:[2]

```
R> fahrenheit_to_celsius ← function(temp_F) {
+   temp_C ← (temp_F - 32) * (5/9)
+   return(temp_C)
+ }
```

[1] Yes, a function to create functions. 🤯

[2] Courtesy of Software Carpentry.

Basic syntax

Writing your own function in R is easy with the `function()` function[1]. The basic syntax is as follows:

```
R> my_func ← function(ARGUMENTS) {
+     OPERATIONS
+     return(VALUE)
+   }
```

Let's try it out with a simple example function - one that converts temperatures from Fahrenheit to Celsius:[2]

```
R> fahrenheit_to_celsius ← function(temp_F) {
+   temp_C ← (temp_F - 32) * (5/9)
+   return(temp_C)
+ }
```

- Our function has an intuitive name.
- Also, it takes just one thing as input, which we call `temp_F`.

[1] Yes, a function to create functions. 🤯

[2] Courtesy of Software Carpentry.

Basic syntax

Writing your own function in R is easy with the `function()` function[1]. The basic syntax is as follows:

```
R> my_func ← function(ARGUMENTS) {
+     OPERATIONS
+     return(VALUE)
+   }
```

Let's try it out with a simple example function - one that converts temperatures from Fahrenheit to Celsius:[2]

```
R> fahrenheit_to_celsius ← function(temp_F) {
+   temp_C ← (temp_F - 32) * (5/9)
+   return(temp_C)
+ }
```

- We now take up the argument `temp_F`, do something with it, and store the output in a new object, `temp_C`.
- Importantly, that object only lives within the function. When the function is run, we cannot access it from the environment.

[1] Yes, a function to create functions. 🤯

[2] Courtesy of Software Carpentry.

Basic syntax

Writing your own function in R is easy with the `function()` function[1]. The basic syntax is as follows:

```
R> my_func ← function(ARGUMENTS) {
+     OPERATIONS
+     return(VALUE)
+   }
```

Let's try it out with a simple example function - one that converts temperatures from Fahrenheit to Celsius:[2]

```
R> fahrenheit_to_celsius ← function(temp_F) {
+   temp_C ← (temp_F - 32) * (5/9)
+   return(temp_C)
+ }
```

- Finally, the output is returned.

[1] Yes, a function to create functions. 🤯

[2] Courtesy of Software Carpentry.

Basic syntax

Writing your own function in R is easy with the `function()` function[1]. The basic syntax is as follows:

```
R> my_func ← function(ARGUMENTS) {
+     OPERATIONS
+     return(VALUE)
+   }
```

Let's try it out with a simple example function - one that converts temperatures from Fahrenheit to Celsius:

```
R> fahrenheit_to_celsius ← function(temp_F) {
+   temp_C ← (temp_F - 32) * (5/9)
+   return(temp_C)
+ }
```

Now, let's try out the function:

```
R> fahrenheit_to_celsius(451)
```

```
  [1] 232.7778
```

Pretty hot, isn't it?

[1] Yes, a function to create functions. 🤯

Functions: default argument values, if(), else()

Let's make the function a bit more complex, but also more fun.

```
R> temp_convert ←
+   function(temp, from = "f") {
+   if (!(from %in% c("f", "c"))){
+     stop("No valid input
+           temperature specified.")
+   }
+   if (from == "f") {
+     out ← (temp - 32) * (5/9)
+   } else {
+     out ← temp * (9/5) + 32
+   }
+   if((from == "c" & temp > 30) |
+      (from == "f" & out > 30)) {
+     message("That's damn hot!")
+   }else{
+     message("That's not so hot.")
+   }
+   return(out) # return temperature
+ }
```

Functions: default argument values, if(), else()

Let's make the function a bit more complex, but also more fun.

- By giving `from` a default value (`"f"`), we ensure that the function returns valid output when only the key input, `temp`, is provided.

```
R> temp_convert ←
+   function(temp, from = "f") {
+   if (!(from %in% c("f", "c"))){
+     stop("No valid input
+           temperature specified.")
+   }
+   if (from == "f") {
+     out ← (temp - 32) * (5/9)
+   } else {
+     out ← temp * (9/5) + 32
+   }
+   if((from == "c" & temp > 30) |
+      (from == "f" & out > 30)) {
+     message("That's damn hot!")
+   }else{
+     message("That's not so hot.")
+   }
+   return(out) # return temperature
+ }
```

Functions: default argument values, if(), else()

Let's make the function a bit more complex, but also more fun.

- By giving `from` a default value (`"f"`), we ensure that the function returns valid output when only the key input, `temp`, is provided.
- `if() { ... }` allows us to make conditional statements. Here, we test for the validity of the input for argument `from`.

```
R> temp_convert ←
+   function(temp, from = "f") {
+   if (!(from %in% c("f", "c"))){
+     stop("No valid input
+           temperature specified.")
+   }
+   if (from == "f") {
+     out ← (temp - 32) * (5/9)
+   } else {
+     out ← temp * (9/5) + 32
+   }
+   if((from == "c" & temp > 30) |
+      (from == "f" & out > 30)) {
+     message("That's damn hot!")
+   }else{
+     message("That's not so hot.")
+   }
+   return(out) # return temperature
+ }
```

Functions: default argument values, if(), else()

Let's make the function a bit more complex, but also more fun.

- By giving `from` a default value (`"f"`), we ensure that the function returns valid output when only the key input, `temp`, is provided.
- `if() { ... }` allows us to make conditional statements. Here, we test for the validity of the input for argument `from`.
- If the condition is not met, the function breaks and prints a message.

```
R> temp_convert ←
+   function(temp, from = "f") {
+   if (!(from %in% c("f", "c"))){
+     stop("No valid input
+           temperature specified.")
+   }
+   if (from == "f") {
+     out ← (temp - 32) * (5/9)
+   } else {
+     out ← temp * (9/5) + 32
+   }
+   if((from == "c" & temp > 30) |
+      (from == "f" & out > 30)) {
+     message("That's damn hot!")
+   }else{
+     message("That's not so hot.")
+   }
+   return(out) # return temperature
+ }
```

Functions: default argument values, if(), else()

Let's make the function a bit more complex, but also more fun.

- By giving `from` a default value (`"f"`), we ensure that the function returns valid output when only the key input, `temp`, is provided.
- `if() { ... }` allows us to make conditional statements. Here, we test for the validity of the input for argument `from`.
- If the condition is not met, the function breaks and prints a message.
- With `else()`, we specify what to do if the `if()` condition is not met.

```
R> temp_convert ←
+   function(temp, from = "f") {
+   if (!(from %in% c("f", "c"))){
+     stop("No valid input
+           temperature specified.")
+   }
+   if (from == "f") {
+     out ← (temp - 32) * (5/9)
+   } else {
+     out ← temp * (9/5) + 32
+   }
+   if((from == "c" & temp > 30) |
+      (from == "f" & out > 30)) {
+     message("That's damn hot!")
+   }else{
+     message("That's not so hot.")
+   }
+   return(out) # return temperature
+ }
```

Functions: default argument values, if(), else()

Let's make the function a bit more complex, but also more fun.

- By giving `from` a default value (`"f"`), we ensure that the function returns valid output when only the key input, `temp`, is provided.
- `if() { ... }` allows us to make conditional statements. Here, we test for the validity of the input for argument `from`.
- If the condition is not met, the function breaks and prints a message.
- With `else()`, we specify what to do if the `if()` condition is not met.
- Make R more talkative with `message()`. Future-You will like it!

```
R> temp_convert ←
+   function(temp, from = "f") {
+   if (!(from %in% c("f", "c"))){
+     stop("No valid input
+           temperature specified.")
+   }
+   if (from == "f") {
+     out ← (temp - 32) * (5/9)
+   } else {
+     out ← temp * (9/5) + 32
+   }
+   if((from == "c" & temp > 30) |
+      (from == "f" & out > 30)) {
+     message("That's damn hot!")
+   }else{
+     message("That's not so hot.")
+   }
+   return(out) # return temperature
+ }
```

Anonymous functions

In R, functions are objects in their own right. They aren't automatically bound to a name. If you choose not to give the function a name, you get an **anonymous function**. You use an anonymous function when it's not worth the effort to give it a name.

Examples:

```
R> map(char_vec, function(x) paste(x, collapse = "|"))
R> integrate(function(x) sin(x) ^ 2, 0, pi)
```

Anonymous functions

In R, functions are objects in their own right. They aren't automatically bound to a name. If you choose not to give the function a name, you get an **anonymous function**. You use an anonymous function when it's not worth the effort to give it a name.

As of `R 4.1.0`, there's a new shorthand syntax for anonymous functions: `\(x)`.

Example:

```
R> (function (x) {paste(x, 'is awesome!')})('Data science') # old syntax
```

```
  [1] "Data science is awesome!"
```

```
R> (\(x) {paste(x, 'is awesome!')})('Data science') # new syntax
```

```
  [1] "Data science is awesome!"
```

Anonymous functions

In R, functions are objects in their own right. They aren't automatically bound to a name. If you choose not to give the function a name, you get an **anonymous function**. You use an anonymous function when it's not worth the effort to give it a name.

As of `R 4.1.0`, there's a new shorthand syntax for anonymous functions: `\(x)`. This plays along nicely with the (native) pipe when we want to pass content to the RHS but not to the first argument.

Anonymous functions

In R, functions are objects in their own right. They aren't automatically bound to a name. If you choose not to give the function a name, you get an **anonymous function**. You use an anonymous function when it's not worth the effort to give it a name.

As of `R 4.1.0`, there's a new shorthand syntax for anonymous functions: `\(x)`. This plays along nicely with the (native) pipe when we want to pass content to the RHS but not to the first argument.

Example:

```
R> mtcars |> subset(cyl == 4) |> (\(x) lm(mpg ~ disp, data = x))()
```

`...` (Dot-dot-dot)

Functions can have a special argument `...` (pronounced *dot-dot-dot*). In other programming languages, this type of argument is often called varargs (short for variable arguments), or ellipsis. With it, a function can take any number of additional arguments. That is potentially very powerful!

A common application is to use `...` to pass those additional arguments on to another function.

Toy example:

```
R> my_list_generator ← function(y, z) {
+   list(y = y, z = z)
+ }
R>
R> my_list_generator_2 ← function(x, ... ) {
+   my_list_generator( ... )
+ }
R>
R> str(my_list_generator_2(x = 1, y = 2, z = 3))
```

```
  List of 2
   $ y: num 2
   $ z: num 3
```

Real-life example:

```
R> map(.x, .f, ... )
R> map(mtcars, mean, na.rm = TRUE)
```

Arguments:

- `.x`: A list or atomic vector
- `.f`: A function
- `...`: Additional arguments passed on to the mapped function.

Writing functions with ChatGPT

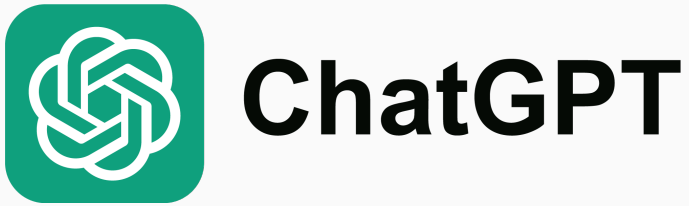
Not every function you plan to write is unique, nor is every problem you want to solve functionally.

ChatGPT, GitHub Copilot and other AI-based coding tools can help you a lot in finding functional solutions you can describe but not verbalize (yet).

I encourage you to use AI for this purpose, but be aware of the necessity to (a) debug and (b) assign credit where due.

Let's try it out with one of the following prompts:

- *Write an R function that capitalizes the first letter of each word in a character vector.*
- *Write an R function that allows me to play one round of black jack.*



Project management

Taming chaos

In the data science workflow, there are two sorts of **surprises** and cognitive stress:

1. Analytical (often good)
2. Infrastructural (almost always bad)

Analytical surprise is when you learn something from or about the data.

Infrastructural surprise is when you discover that:

- You can't find what you did before.
- The analysis code breaks.
- The report doesn't compile.
- The collaborator can't run your code.

Good project management lets you focus on the right kind of stress.

Keeping Future-you happy

- It’s often tempting to set up a project assuming that you will be the only person working on it, e.g. as homework.
- That’s almost never true.
- Coauthors and collaborators happen to the best of us.
- Even if not, there's someone else who you always have to keep happy: Future-you.
- Future-you is really the one you organize your projects for.
- They are who you use version control for (see later).
- Most importantly, they are who will enjoy the fruits of your data science labor, or have to fight back your chaos.
- So, be kind to Future-you. Establish a good workflow. You'll thank yourself later.



Project setup

You should **always** think in terms of projects.

A project is a **self-contained unit of data science work** that can be

- Shared
- Recreated by others
- Packaged
- Dumped

A project contains

- Content, e.g., raw data, processed data, scripts, functions, documents and other output
- Metadata, e.g., information about tools for running it (required libraries, compilers), version history

For R projects

- Projects are folders/directories.
- Metadata is the RStudio project (`.Rproj`) files (perhaps augmented with the output of renv for dependency management) and `.git`.

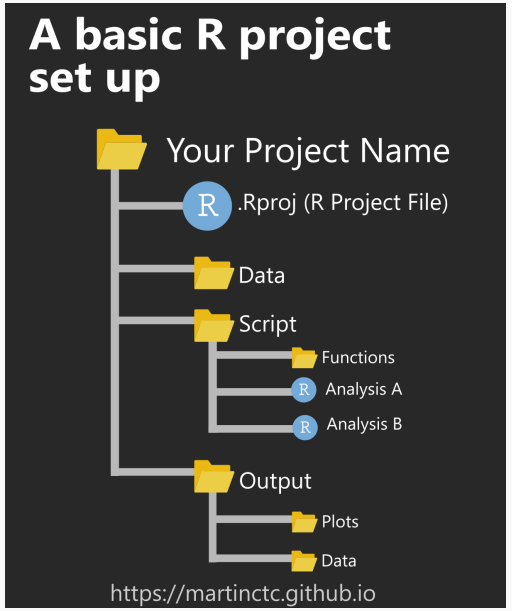
Setup: the folder structure

Structuring your working directory

- One folder contains everything inside it.
- Directories keep things separate that should be separated.
- You decide on the fundamental structure. The project decides on the details.

Further thoughts

- Ideally, your project folder can be relocated without problem.
- Keep input separate from output. Definitely separate raw from processed data!
- Structure should be capable of evolution. More data, cases, models, output formats shouldn't be a problem.




```
.
└── my_awesome_project
    ├── src
    ├── output
    ├── data
    │   ├── raw
    │   └── processed
    ├── README.md
    ├── run_analyses.R
    └── .gitignore
```

`Credit` Chris/r-bloggers.com

Setup: the paths

Good paths

- All internal paths are relative.
- They are invariant to moving/sharing the project.
- Examples:
  - `"preprocessing.R"`
  - `"figures/model-1.png"`
  - `"../data/survey.RDa"`

Bad paths

- Using `setwd()` is bad practice 99% of the times.
- Absolute paths are bad paths. Don't feed functions with paths like `"/Users/me/data/thing.sav"`.
- Those paths will not work outside your computer (or maybe not even there, some days/weeks/months ahead).

The working directory

- Set it manually once per session (do `Session > Set Working Directory > Choose Directory`). Then all your good paths will "just work".
- Better yet, get it right automatically by opening RStudio with clicking on the script you want to work with. This will set the location of the script as working directory (which should be your working assumption, too).
- Even better yet, have the metadata set it for you:
  - Open your session by opening (choosing, clicking on) `myproject.Rproj`
  - Then you'll get the path set for you.
- That's probably better than the previous option because you might not want your `code` directory to be the working directory.

Setup: the code structure

Naming scripts

- Files should have short, descriptive names that indicate their purpose.
- I recommend the use of telling verbs.
- Names should only include letters and numbers with dashes `-` or underscores `_` to separate words.
- Use numbering to indicate the order in which files should be run:
  - `0-setup.R`
  - `1-import-data.R`
  - `2-preprocess-data.R`
  - `3-describe-uptake.R`
  - `4-analyze-uptake.R`
  - `5-analyze-experiment.R`

Modularizing scripts

- Write short, modular scripts. Every script serves a purpose in your pipeline.
- This makes things easier to debug.
- At the beginning of a script you might want to document input and output.

Setup: the code structure (cont.)

Talk to Future-you

- Describe your code, e.g. by starting with a description of what it does. If you comment/describe a lot, consider using an R Markdown (`.Rmd`) file instead of a simple `.R` script.
- Put the setup first (e.g., `library()` and `source()`).
- You might want to outsource the loading of packages to a separate script that is imported in the first step (`source("functions.R")`) or just declared the first script in the pipeline.
- Always comment more than you usually do.

Structuring your code

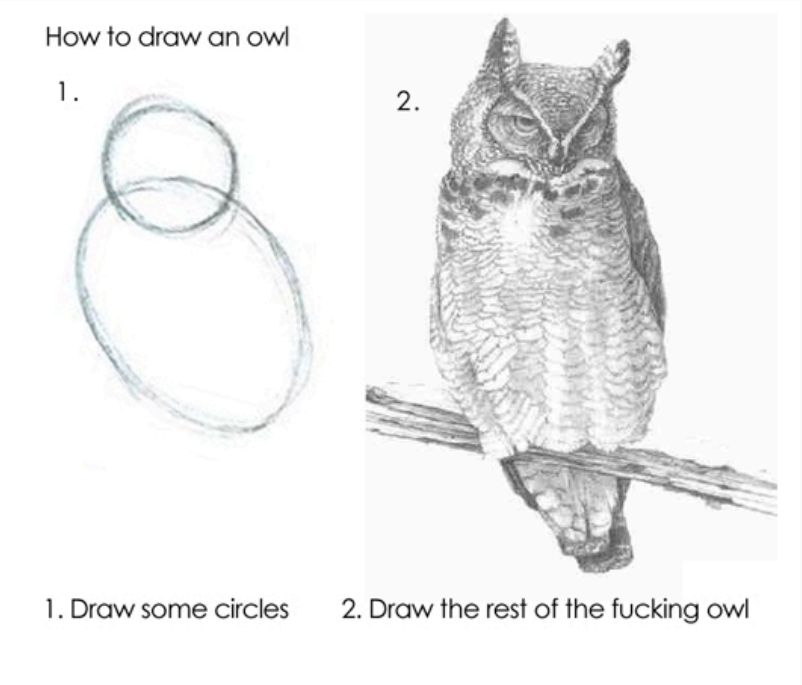
- Even with modularized code, scripts can become long. Structure helps to keep an overview.
- Use commented lines as section/subsection heads.
- RStudio creates a "table of contents" when you name your code chunks as follows (`#` followed by title and `---`):

```
R> # Import data --------------
R>
R> dat ← read_csv("dat.csv")
```

Setup: the rest

More things to consider

- There'd be more to say on how to establish a good project workflow, including how to
  - store/organize raw and derived data,
  - deal with output in form of graphs and tables,
  - link everything together from start (project setup) to finish (knitting the report)
  - separate coding for the record and experimental coding.
- But there's limited value in teaching you all that upfront.
- The truth is: You'll likely refine your own workflow over time. I just saved you some initial pain (hopefully).
- Do check out other people's experiences and opinions, e.g., here or here.



Managing your R project in two simple steps

Coding style

Coding style: the basics

Why adhere to a particular style of coding?

- It reduces the number of arbitrary decisions you have to consciously make during coding. We make an arbitrary decision (convention) once, not always ad hoc.
- It provides consistency.
- It makes code easier to write.
- It makes code easier to read, especially in the long term (i.e. two days after you've closed a script).

What are questions of style?

- Questions of style are a matter of opinion.
- We will mostly follow Hadley Wickham’s opinion as expressed in the "tidyverse style guide".
- We'll consider how to
  - name,
  - comment,
  - structure, and
  - write.

Naming things

Surprisingly many things can go wrong with naming...

"There are only two hard things in Computer Science: cache invalidation and naming things." - *Phil Karlton*

`Credit` karlton.org

`Credit` Mashable

Naming files

- Code file names should be meaningful and end in `.R`.
- Avoid using special characters in file names. Stick with numbers, letters, dashes (`-`), and underscores (`_`).
- Some examples:

```
# Good
fit_models.R
utility_functions.R

# Bad
fit models.R
foo.r
stuff.r
```

- If files should be run in a particular order, prefix them with numbers:

```
00_download.R
01_explore.R
...
09_model.R
10_visualize.R
```

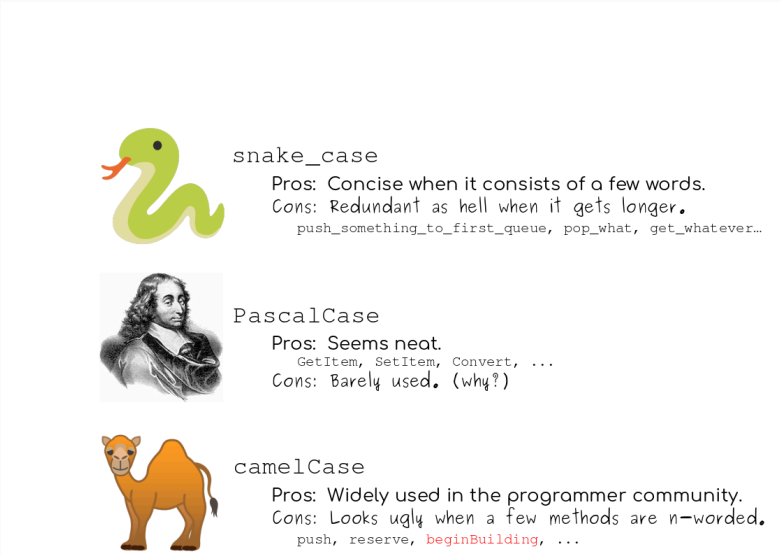
Naming objects and variables

- There are various conventions of how to write phrases without spaces or punctuation. Some of these have been adapted in programming, such as camelCase, PascalCase, or snake_case.
- The `tidyverse` way: Object and variable names should use only lowercase letters, numbers, and underscores.
- Examples:

```
# Good
day_one # snake_case
day_1 # snake_case

# Less good
dayOne # camelCase
DayOne # PascalCase
day.one # dot.case

# Dysfunctional
day-one # kebab-case
```



`Credit` cassert24/Reddit

Naming objects and variables *cont.*

Table 1:

File folder structure for organizing model and analysis files used in the proposed DARTH coding framework.

Folder name	Folder function
data-raw	This is where raw data is stored alongside ".R" scripts that read in raw data, process these data, and calls use_this::use_data(<processed data>) to save .rda formatted data files in the "data" folder. These data could include a ".csv" file with input parameters derived from the published literature, as well as internal R data files (with .RData, .rds, or .rda extensions) containing primary data from which model input values will be estimated through statistical models embedded into the analysis.
data	This is where input data is stored to be used in the different components of the CEA. These data could be generated from raw data stored in the "data-raw" folder. Essentially, this folder stores the cleaned or processed versions of raw data that has been gathered from elsewhere
R	This is where ".R" files that define functions to be used as part of the analysis are stored. These are functions that are specific to the analysis. The model will be one such function; however, other functions will likely be used, such as computing the fit of the model output to the specific calibration targets of the analysis. This folder also stores ".R" scripts that document the datasets in the "data" folder.
analysis	This is where interactive scripts of the analysis would be stored. These scripts control the overall flow of the analysis. This is also where many operations that ultimately become functions will be developed and debugged.
output	This is where output files of the analysis should be stored. These files may be internal R data files (".RData", ".rds", ".rda") or external data files (such as ".csv"). Examples of files stored here would be the output of the model calibration component or the PSA dataset generated in the uncertainty analysis component. These data files can then be loaded by other components without having to first rerun previous components (e.g. the calibrated model values can be loaded for a base case analysis without re-running the calibration).
figs	For analyses that will include figures, we generally create a separate figures folder. Though these could be stored in the output folder, it can be helpful to have a separate folder so that the images of the figure files can be easily previewed. This is particularly important for analyses that generate a large number of figures.
tables	This folder includes tables to be included in a publication or report, such as the table of intervention costs and effects and ICERs.
report	A report folder could be used to store R Markdown files to describe in detail the model-based CEA by using all the functions and data of the framework, run analyses and display figures. The R Markdown files can be compiled into .html, .doc or .pdf files to generate a report of the CEA. This report could be the document submitted to HTA agencies accompanying the R code of the model-based CEA.
vignettes	A vignettes folder could be used to describe the usage of the functions and data of each of some or all components of the framework through accompanying R Markdown files as documentation. The R Markdown file can use all the functions, outputs, and figures to integrate the R code into the Markdown text.
tests	A tests folder includes ".R" scripts that runs all the unit tests of the functions in the framework. A good practice is to have one file of tests for each complicated function or for each of the components of the framework.

`Credit` Alarid et al. 2019

Table 2.

File and variable naming conventions in the proposed DARTH coding framework.

Object type	Naming recommendation	Examples
Files	dir/<component number>_<description>.<ext>	• analysis/θ1_model_inputs.R • R/θ2_simulation_model_functions.R
Functions	<action!>_<description>	• generate_init_params() • generate_psa_params()
Variables	<x>_<y>_<var_name> where x = data type prefix y = variable type prefix var_name = brief descriptor	• n_samp • hr_S1D • v_r_mort_by_age • a_M • l_params_all • df_out_ce

Table 3.

Recommended prefixes in variable names that encode data and variable type.

Prefix	Data type	Prefix	Variable type
<> (no prefix)	scalar	n	number
v	vector	p	probability
m	matrix	r	rate
a	array	u	utility
df	data frame	c	cost
dtb	data table	hr	hazard ratio
l	list	rr	relative risk

Naming functions

- In addition to following the general advice for object names, strive to use verbs for function names:

```
# Good
add_row()
permute()

# Bad
row_adder()
permutation()
```

- Also, try avoiding function names that already exist, in particular those that come with a loaded package.
- This often implies a trade-off between shortness and uniqueness. In any case, you would try to avoid situations that force you disambiguate functions with the same name (as in `dplyr::select`; see "R packages").
- Check out this Wikipedia page) or this Stackoverflow post for more background on naming conventions in programming!
- For more good advice on how to name stuff, see this legendary presentation by Jenny Bryan.

Commenting on things

Why comment at all?

- It’s often tempting to set up a project assuming that you will be the only person working on it, e.g. as homework. But that's almost never true.
- You have project partners, co-authors, principals.
- Even if not, there's someone else who you always have to keep happy: Future-you.
- Comment often to make Future-you happy about Past-you by document what Present-You is doing/thinking/planning to do.

Past-you

Present-you

Future-you

Commenting on things *cont.*

General advice

- Each line of a comment should begin with the comment symbol and a single space: `#`
- Use comments to record important findings and analysis decisions.
- If you need comments to explain what your code is doing, consider rewriting your code to be clearer.
- But: comments can work well as "sub-headlines".
- If you discover that you have more comments than code, consider switching to R Markdown.
- (Longer) comments generally work better if they get their own line.

```
R> # define job status
R> dat$at_work ← dat$job %in% c(2, 3)
R> dat$at_work ← dat$job %in% c(2, 3) # define job
```

Giving structure

- Use commented lines together with dashes to break up your file into easily readable chunks.
- RStudio automatically detects these chunks and turns them into sections in the script outline.

```
R> # Input/output --------------------
R>
R> # input
R> c("data/survey2021.csv")
R>
R> #  output
R> c("survey_2021_cleaned.RData",
+   "resp_ids.csv")
R>
R> # Load data ----------------------
R>
R> # Plot data ----------------------
```

Other stuff

- Use **spaces** generously, but not too generously. Always put a space after a comma, never before, just like in regular English.
- Use `←`, not `=`, for **assignment**.
- For **logical operators**, prefer `TRUE` and `FALSE` over `T` and `F`.
- To facilitate readability, **keep your lines short**. Strive to limit your code to about 80 characters per line.
- If a **function call is too long** to fit on a single line, use one line each for the function name, each argument, and the closing bracket.
- Use **pipes**. When you use them, they should always have a space before it, and should usually be followed by a new line.

Spacing

```
R> # Good
R> mean(x, na.rm = TRUE)
R> height ← (feet * 12) + inches
R>
R> # Bad
R> mean(x,na.rm=TRUE)
R> mean ( x, na.rm = TRUE )
R> height←feet*12+inches
```

Piping

```
R> babynames %>%
+   filter(name %>% equals("Kim")) %>%
+   group_by(year, sex) %>%
+   summarize(total = sum(n)) %>%
+   qplot(year, total, color = sex, data = .,
+         geom = "line") %>%
+   add(ggtitle('People named "Kim"')) %>%
+   print
```

Next steps

Assignment

Assignment 1 is online! You have a bit more than a week to work on it - final uploading deadline is Sep 23, 9:30am CET.

Next lecture

Programming II Buckle up and bring coffee, because it'll get both exciting and tedious at the same time. We're going to iterate, automate, schedule, and debug.